

Rapport technique

Data Lake financier

Groupe : Artemiy Smogunov, Nathan Smadja-Tubiana, Patrice Ignongui

Projet EFREI 2025-2026

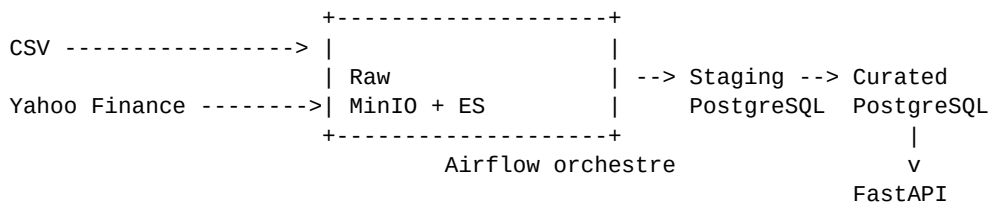
Rapport technique du Data Lake financier

Périmètre

Ce projet met en place un pipeline de données financières avec deux sources, trois zones de traitement, une orchestration Airflow et une API de consultation. Nous avons retenu un périmètre local reproductible avec Docker Compose. Le projet ne contient pas d'interface métier dédiée, de moteur de recommandation financière ni de déploiement cloud.

Les deux sources sont le fichier versionné `finance_dataset.csv` et Yahoo Finance, interrogé avec `yfinance`. Le CSV contient 732 lignes AAPL du 3 janvier 2022 au 29 novembre 2024. Le DAG interroge aussi dix actions et quatre indices pour récupérer le dernier cours disponible.

Architecture exécutée



PostgreSQL 15 contient les tables Staging, Curated, les journaux des ingestions manuelles et les tables internes Airflow.

MinIO conserve les objets Raw. Le bucket `raw-financial-data` reçoit les CSV. Le bucket `raw-api-data` reçoit les payloads JSON de l'ingestion planifiée.

Elasticsearch 8.11.0 contient l'index `raw_financial_events`. Chaque document possède un identifiant construit avec la source, le ticker et la date.

Airflow 2.8.1 utilise `LocalExecutor`. Le Webserver expose l'interface et le scheduler lance les tâches.

FastAPI expose les routes de santé, de statistiques, de lecture et d'ingestion manuelle. Uvicorn démarre quatre workers dans le conteneur API.

Reproductibilité Python avec uv

Le dépôt possède maintenant un `pyproject.toml`, un `.python-version` et un `uv.lock` à la racine. Python est fixé à la branche 3.11. Le verrou contient les versions transitives exactes.

Les dépendances métier communes sont Elasticsearch, MinIO, NumPy, pandas, psycopg2, scikit-learn et yfinance. Les dépendances FastAPI, tests et PDF sont séparées dans des groupes uv.

L'image API exécute `uv sync --frozen`. L'image Airflow est construite une seule fois avant le démarrage des services. Elle ne repose plus sur `_PIP_ADDITIONAL_REQUIREMENTS`, qui installait les paquets dans `airflow-init`, le Webserver et le scheduler à chaque création de conteneur.

Airflow 2.8.1 impose des versions internes précises de rich, protobuf et markdown-it-py. L'image Airflow garde ces trois versions. Les autres dépendances métier viennent du verrou. Le build exécute `pip check` et échoue si l'environnement contient une incompatibilité.

La validation finale donne les mêmes versions métier dans l'API et Airflow : Python 3.11, yfinance 1.4.1, pandas 2.2.0, NumPy 1.26.4, scikit-learn 1.4.0 et le client Elasticsearch 8.12.1.

Organisation du code

Le code est réparti entre config, ingestion, transformation, api, airflow, scripts et tests.

La proposition d'un package sous src n'est pas implémentée dans cette version. Les imports actuels, les volumes Airflow et le chemin par défaut du CSV dépendent de l'organisation présente. Une migration partielle aurait rendu la documentation fausse et risqué de casser le DAG. Le code reste donc réutilisable à l'intérieur du dépôt, mais il n'est pas publié comme package Python installable.

Ingestion du fichier

L'ingestion fichier lit le CSV, normalise les noms de colonnes, convertit les valeurs numériques et vérifie la présence de ticker, date, open, high, low, close et volume.

Le ticker est mis en majuscules. La date est convertie au format YYYY-MM-DD. Les lignes sans ticker, date ou cours de clôture sont retirées. Pour une clé (ticker, date) répétée, la dernière ligne est conservée.

Le jeu de données est ensuite séparé par ticker. Chaque partie est écrite en CSV dans MinIO et indexée par lot dans Elasticsearch avec la source `yfinance_file`.

Ingestion Yahoo Finance

L'ingestion API calcule une date de début à partir du nombre de jours demandé. Pour chaque ticker, `yfinance.Ticker.history` récupère les cours.

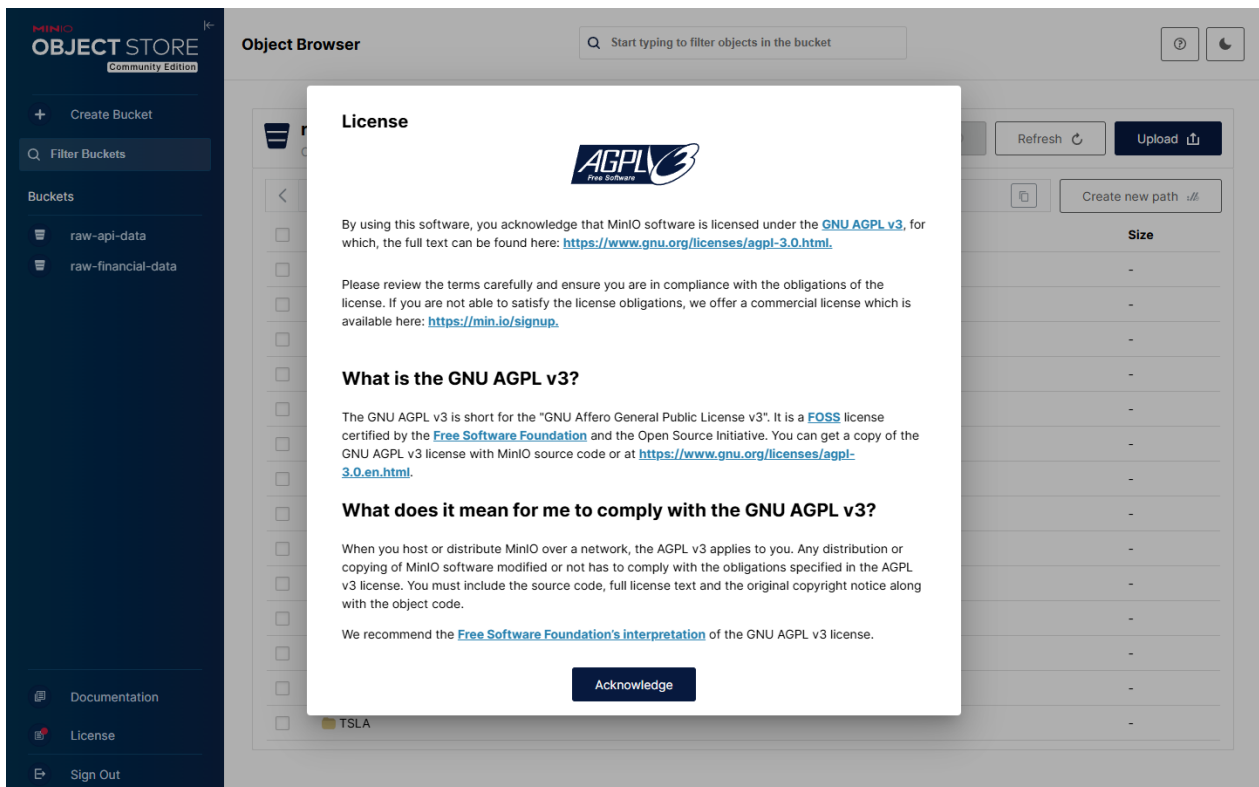
Le payload contient le ticker, l'heure de collecte, la source, les cours et des métadonnées lorsqu'elles sont disponibles : nom court, secteur, industrie, capitalisation, devise et place de cotation.

Le JSON complet est écrit dans MinIO. Les cours sont ensuite indexés dans Elasticsearch avec la source `yfinance_api`.

Le DAG utilise une fenêtre de deux jours. Lors du run final, chaque ticker a retourné une ligne datée du 10 juillet 2026. Les quatorze tickers ont réussi.

Zone Raw

MinIO conserve le contenu des sources dans leur format de transport. Elasticsearch rend les lignes interrogeables avec des filtres sur le ticker, la source et la date.



Buckets et contenu API dans MinIO

La capture montre les deux buckets et les dossiers créés pour les tickers API. Un run sur volumes vides produit un objet fichier et quatorze objets API.

```
{
  "total": 733,
  "returned": 1,
  "documents": [
    {
      "ticker": "AAPL",
      "date": "2026-07-10",
      "open": 314.720001220703,
      "high": 316.910003662109,
      "low": 312.170013427734,
      "close": 315.320007324219,
      "volume": 34109200,
      "source": "yfinance_api",
      "metadata": {
        "shortName": "Apple Inc.",
        "sector": "Technology",
        "industry": "Consumer Electronics",
        "marketCap": 4631217307648,
        "currency": "USD",
        "exchange": "NYS"
      },
      "ingested_at": "2026-07-12T07:59:55.428306+00:00"
    }
  ]
}
```

Dernière ligne AAPL dans Raw

La ligne Raw du 10 juillet 2026 provient de `yfinance_api`. Elle contient le cours, le volume et les métadonnées Apple renvoyées au moment du test.

Transformation Staging

Staging lit les documents Elasticsearch d'un ticker par date croissante. Le code récupère au maximum 10 000 documents pour un ticker.

La préparation effectue les traitements suivants :

- conversion du ticker en majuscules ;
- conversion de la date et des valeurs numériques ;
- remplacement d'un volume manquant par zéro ;
- tri par ticker et date ;
- conservation d'une ligne par (ticker, date) ;
- suppression des lignes sans cours de clôture.

Les indicateurs calculés sont :

- SMA 20 et SMA 50 ;
- EMA 12 et EMA 26 ;
- RSI 14 ;
- MACD et ligne de signal ;
- bandes de Bollinger sur 20 périodes ;
- rendement par rapport à la ligne précédente ;
- volatilité glissante sur 20 périodes.

Les lignes sont écrites dans `staging_ohlcv` avec un `upsert`. Un nouveau run peut donc corriger une date sans créer de doublon.

Impression élégante ☒

```
{
  "total": 733,
  "returned": 1,
  "offset": 0,
  "limit": 1,
  "data": [
    {
      "ticker": "AAPL",
      "date": "2026-07-10",
      "open": "314.720001",
      "high": "316.910004",
      "low": "312.170013",
      "close": "315.320007",
      "adj_close": null,
      "volume": 34109200,
      "sma_20": "232.381783",
      "sma_50": "230.455832",
      "ema_12": "244.307807",
      "ema_26": "235.898777",
      "rsi_14": "92.2640",
      "macd": "8.409029",
      "macd_signal": "2.210671",
      "bollinger_upper": "272.366227",
      "bollinger_lower": "192.397338",
      "daily_return": "0.328614",
      "volatility_20": "0.073243",
      "ingested_at": "2026-07-12 08:00:16.427784"
    }
  ]
}
```

AAPL après la transformation Staging

Cette capture montre les indicateurs calculés pour la dernière ligne AAPL.

Transformation Curated

Curated relit l'historique Staging du ticker. Le code calcule d'abord un z-score glissant du volume sur 20 lignes, avec un minimum de cinq observations.

Isolation Forest utilise quatre variables : rendement journalier, volatilité sur 20 périodes, z-score du volume et RSI. Les valeurs manquantes sont remplacées par la médiane de leur colonne pour l'entrée du modèle.

Le modèle ne s'exécute pas sous 30 lignes. Au-dessus de ce seuil, il utilise 100 estimateurs, une contamination de 5 %, `random_state=42` et tous les cœurs disponibles.

Les anomalies détectées sont classées dans cet ordre :

- rendement inférieur à -5 % : `flash_crash` ;
- rendement supérieur à 5 % : `price_spike` ;
- z-score absolu du volume supérieur à 3 : `volume_spike` ;
- volatilité supérieure à 4 % : `high_volatility` ;
- autre point isolé : `unknown_anomaly`.

La tendance vaut `bullish` lorsque le cours est supérieur à la SMA 20, elle-même supérieure à la SMA 50. La condition inverse produit `bearish`. Les autres cas produisent `neutral`.

Le signal vaut `buy` lorsque le RSI est inférieur à 30 et que le MACD est au-dessus de sa ligne de signal. Il vaut `sell` lorsque le RSI dépasse 70 et que le MACD est sous sa ligne de signal. Sinon, il vaut `hold`.

```
{
  "total": 733,
  "returned": 1,
  "offset": 0,
  "limit": 1,
  "data": [
    {
      "ticker": "AAPL",
      "date": "2026-07-10",
      "close": "315.320007",
      "volume": 34109200,
      "daily_return": "0.328614",
      "volatility_20": "0.073243",
      "rsi_14": "92.2640",
      "anomaly_score": "-0.740538",
      "is_anomaly": true,
      "anomaly_type": "price_spike",
      "price_trend": "bullish",
      "signal": "hold",
      "processed_at": "2026-07-12 00:00:26.838875"
    }
  ]
}
```

AAPL après la transformation Curated

La dernière ligne AAPL est une anomalie `price_spike`, avec une tendance `bullish` et un signal `hold`.

Discontinuité temporelle observée

Le rendement de 32,86 % visible sur la dernière ligne ne représente pas une séance boursière. Le dernier point du CSV date du 29 novembre 2024. La ligne API suivante date du 10 juillet 2026. Après le tri, `pct_change` compare ces deux cours malgré l'intervalle de plus d'un an.

Cette rupture augmente aussi la volatilité et influence le score d'anomalie. Le projet ne détecte pas encore les trous de calendrier avant le calcul des indicateurs. La capture est conservée parce qu'elle correspond au comportement réel du code et du dataset.

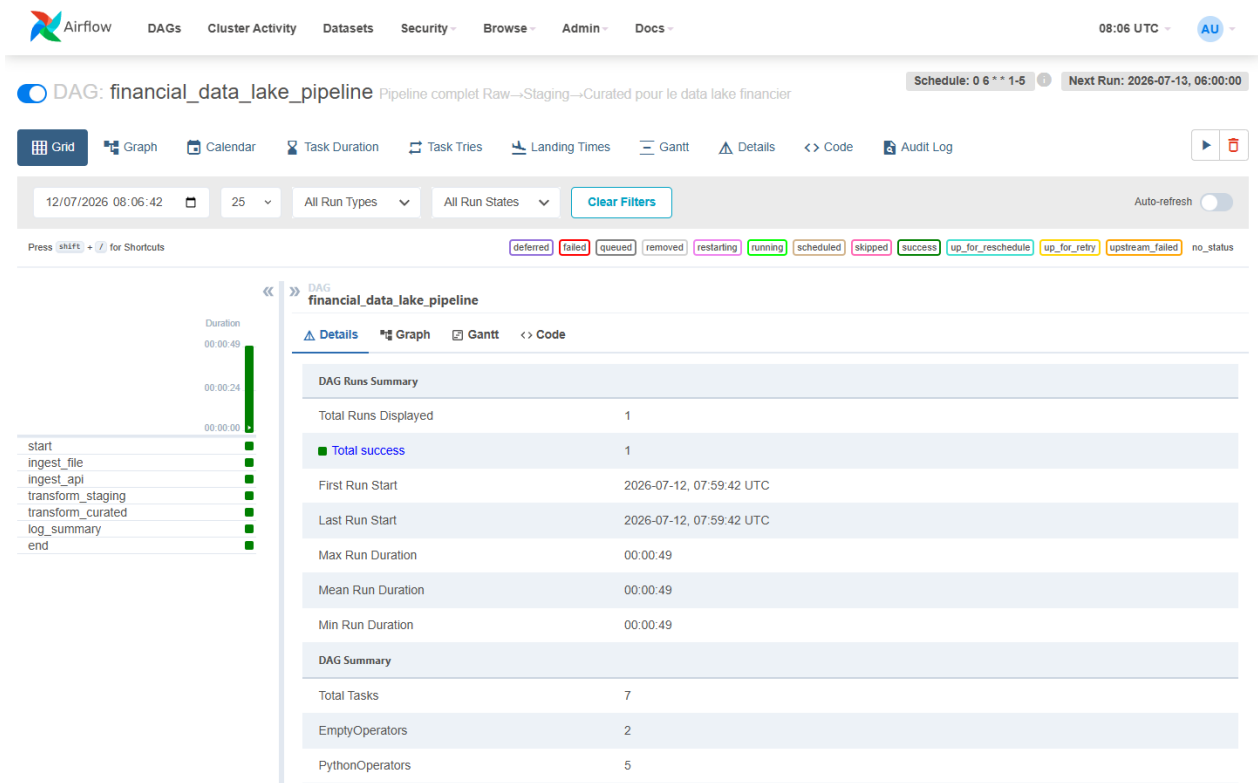
DAG Airflow

Le DAG est planifié à 6 h UTC du lundi au vendredi, avec `catchup=False` et un seul run actif. Chaque tâche peut être retentée deux fois avec un délai de cinq minutes.

Les tâches `ingest_file` et `ingest_api` poussent leur résultat dans XCom. Staging lit ces XCom pour construire la liste des tickers ayant réussi. Curated utilise ensuite la liste des tickers traités par Staging.

Version: v2.8.1
Git Version: .release:c0ffa9c5d96625c68ded9562632674ed366b5eb3

Page 9



Le run `documentation_verified_20260712T075000Z` s'est terminé en 49,3 secondes. Les sept tâches sont en succès.

Le log `log_summary` indique une ingestion fichier réussie, quatorze ingestions API réussies, aucune erreur, 746 lignes Staging, 746 lignes Curated et 37 anomalies.

API

Financial Data Lake API 1.0.0 OAS 3.1

/openapi.json

API Gateway du data lake financier (Yahoo Finance / yfinance). Expose les trois zones du data lake (Raw, Staging, Curated) et fournit deux endpoints d'ingestion.

Health		^
GET	/health	Vérification de l'état des services
Stats		^
GET	/stats	Métriques de remplissage du data lake
Raw		^
GET	/raw	Données brutes de la zone Raw
GET	/raw/objects	Liste des objets MinIO dans la zone Raw
Staging		^
GET	/staging	Données staging avec indicateurs techniques
GET	/staging/tickers	Liste des tickers disponibles en staging
Curated		^
GET	/curated	Données curated avec scores d'anomalie
GET	/curated/anomalies/summary	Résumé des anomalies par ticker
GET	/curated/signals	Tickers avec signaux actifs (buy/sell)
Ingest		^
POST	/ingest	Ingestion synchrone avec benchmark
Ingest Fast		^
POST	/ingest_fast	Ingestion parallèle

Routes exposées dans Swagger

Les routes de lecture utilisent directement les clients PostgreSQL, MinIO et Elasticsearch du module de dépendances API. Le client n'a pas besoin de connaître les schémas ou protocoles propres à ces stockages.

GET /raw lit Elasticsearch. Le paramètre source accepte file, api et manual et les traduit vers les valeurs stockées.

GET /staging et GET /curated utilisent une pagination avec limit et offset. Les dates et les tickers peuvent être filtrés.

POST /ingest traite les tickers un par un. POST /ingest_fast utilise huit threads pour les téléchargements, huit threads pour MinIO, une écriture Elasticsearch groupée et execute_values pour PostgreSQL.

Les deux routes manuelles écrivent la source yfinance_manual dans Elasticsearch. Cette source commune permet au benchmark de comparer les deux implémentations sans donner un sens métier différent aux documents.

Benchmark versionné

Le script de benchmark appelle les deux routes avec un lot de 1 ticker et un lot de 100 tickers. Il effectue trois répétitions et alterne l'ordre des appels pour limiter l'avantage systématique du premier mode.

Le JSON versionné contient les médianes suivantes :

- 1 ticker : 1 073,29 ms en mode standard et 989,86 ms en mode rapide, soit 7,77 % de gain ;
- 100 tickers : 104 825,65 ms en mode standard et 12 230,36 ms en mode rapide, soit 88,33 % de gain.

L'objectif de 30 % est atteint pour 100 tickers, pas pour un ticker. Le fichier de benchmark a été produit avant la validation finale du 12 juillet. Il est conservé comme résultat du test de performance, mais il ne doit pas être confondu avec le run Airflow final.

Tests

La commande `uv run pytest -q` retourne 14 tests réussis.

Les tests utilisent des DataFrames locaux et des mocks pour Elasticsearch. Ils couvrent les calculs financiers et la logique du benchmark. Ils ne démarrent pas Docker et ne testent pas les connexions réelles.

Le test d'intégration est donc séparé : reconstruction des images, volumes vides, `/health`, `pip check`, absence d'erreur d'import Airflow, run complet et lecture des trois zones.

Résultats de la validation finale

La validation a été faite le 12 juillet 2026 après `docker compose down -v` puis `docker compose up -d --build`.

Résultats :

- 14 tests Python réussis ;
- toutes les images construites depuis les fichiers actuels ;
- API et Airflow sans dépendance cassée ;
- PostgreSQL, MinIO et Elasticsearch en état ok ;
- aucune erreur d'import du DAG ;
- 1 source fichier et 14 sources API réussies ;
- 15 objets MinIO ;
- 746 documents Elasticsearch ;
- 746 lignes Staging ;
- 746 lignes Curated ;
- 37 anomalies, soit 4,96 % des lignes Curated ;
- sept tâches Airflow réussies en 49,3 secondes.

```
{
  "raw_minio": {
    "bucket_file_objects": 1,
    "bucket_api_objects": 14,
    "total_objects": 15
  },
  "raw_elasticsearch": {
    "total_documents": 746,
    "by_ticker": {
      "AAPL": 733,
      "AMZN": 1,
      "BRK-B": 1,
      "GOOGL": 1,
      "JNJ": 1,
      "JPM": 1,
      "META": 1,
      "MSFT": 1,
      "NVDA": 1,
      "TSLA": 1,
      "DJIA": 1,
      "GSPC": 1,
      "IXIC": 1,
      "RUT": 1
    }
  },
  "staging": {
    "total_rows": 746,
    "by_ticker": [
      {
        "ticker": "AAPL",
        "rows": 733,
        "from": "2022-01-03",
        "to": "2026-07-10"
      },
      {
        "ticker": "AMZN",
        "rows": 1,
        "from": "2026-07-10",
        "to": "2026-07-10"
      },
      {
        "ticker": "BRK-B",
        "rows": 1,
        "from": "2026-07-10",
        "to": "2026-07-10"
      },
      {
        "ticker": "DJIA",
        "rows": 1,
        "from": "2026-07-10",
        "to": "2026-07-10"
      },
      {
        "ticker": "GOOGL",
        "rows": 1,
        "from": "2026-07-10",
        "to": "2026-07-10"
      },
      {
        "ticker": "GSPC",
        "rows": 1,
        "from": "2026-07-10",
        "to": "2026-07-10"
      },
      {
        "ticker": "IXIC",
        "rows": 1,
        "from": "2026-07-10",
        "to": "2026-07-10"
      },
      {
        "ticker": "JNJ",
        "rows": 1,
        "from": "2026-07-10",
        "to": "2026-07-10"
      },
      {
        "ticker": "JPM"
      }
    ]
  }
}
```

Compteurs Raw et Staging

```

Impression élégante ☒
{
  "rows": 1,
  "from": "2026-07-10",
  "to": "2026-07-10"
},
{
  "ticker": "JPM",
  "rows": 1,
  "from": "2026-07-10",
  "to": "2026-07-10"
},
{
  "ticker": "META",
  "rows": 1,
  "from": "2026-07-10",
  "to": "2026-07-10"
},
{
  "ticker": "MSFT",
  "rows": 1,
  "from": "2026-07-10",
  "to": "2026-07-10"
},
{
  "ticker": "NVDA",
  "rows": 1,
  "from": "2026-07-10",
  "to": "2026-07-10"
},
{
  "ticker": "^RUT",
  "rows": 1,
  "from": "2026-07-10",
  "to": "2026-07-10"
},
{
  "ticker": "TSLA",
  "rows": 1,
  "from": "2026-07-10",
  "to": "2026-07-10"
}
],
"curated": {
  "total_rows": 746,
  "anomalies_detected": 37,
  "anomaly_rate_pct": 4.96,
  "by_ticker": [
    {
      "ticker": "AAPL",
      "rows": 733,
      "anomalies": 37,
      "last_date": "2026-07-10"
    },
    {
      "ticker": "AMZN",
      "rows": 1,
      "anomalies": 0,
      "last_date": "2026-07-10"
    },
    {
      "ticker": "BRK-B",
      "rows": 1,
      "anomalies": 0,
      "last_date": "2026-07-10"
    },
    {
      "ticker": "^DJI",
      "rows": 1,
      "anomalies": 0,
      "last_date": "2026-07-10"
    },
    {
      "ticker": "GOOGL",
      "rows": 1,
      "anomalies": 0,
      "last_date": "2026-07-10"
    },
    {
      "ticker": "^GSPC",

```

Compteurs Curated

Limites

Yahoo Finance reste un service externe. Une panne, une limitation ou une modification de réponse peut produire une erreur par ticker.

Le dataset fichier ne rejoint pas l'historique API de façon continue. Le calcul des rendements ne corrige pas cette rupture.

Les tickers qui ne possèdent qu'une ligne ne passent pas dans Isolation Forest. Le modèle ne travaille réellement que sur AAPL lors du run final.

Le signal technique n'a pas été évalué comme stratégie d'investissement.

Les identifiants sont écrits dans Docker Compose pour simplifier la correction locale. Le CORS de FastAPI accepte toutes les origines. Ce choix ne convient pas à une mise en ligne publique.

Le projet reste organisé en modules à la racine et n'est pas installable comme package src réutilisable par un autre dépôt.